

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	In progress	5
Bug	Operator-blocked	3
Bug	Queued	5
Task	In progress	13
Task	Queued	10
Task	Ready for testing	2
TOTAL		38

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): COMPLETE THE CAPTCHA FLOW ONCE Session establishment path is DECIDED: the operator drives the interactive CAPTCHA flow at /api/v1/auth/rutracker/captcha then /login once, and the proxy stores the resulting bb_session. The copy paste path and the cookies-file autoload path were both offered and NOT chosen. THE BLOWNARROWS BUT DOES NOT LIFT: the operator must physically drive the flow — no agent can solve a CAPTCHA (§11.4.52 honest operator_attended boundary). AGENT PREPARED BEFORE THE HAND-OFF, so the operator's single attempt succeeds rather than discover

ATM ID	Type	Status	Severity	Description
BOB-065	Task	In progress	High	broken endpoint mid-flow: verify both endpoints are reachable and correctly wired end-to-end; verify the session is actually PERSISTED after successful post (not merely accepted), and verify a stored session is actually USED by a subsequent search. Honest limitation to state up front: a stored bb_session expires, so this path requires repeating — that cost was accepted with the decision. This answer is recorded as consumed DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applies to decisions). — prior item text follows — RuTracker automated login blocked by CAPTCHA Lava P2: Egress diagnosis and VPN-host SOCKS routing (containers pkg/egress) === ROUND-2 2026-09-25 — the egress/VPN mechanism ALREADY EXISTS (2026-07-01, commit be5062 submodule cde354f/e273fd2/3a52825), this item's own tracker row was just never reconciled against it === Root cause of the confusion: Two different “containers submodule” paths exist in this tree. constitution/submodules/containers (an unrelated nested dependency, no pkg/ dir) NOT what this item or docs/PORTING-FROM-LAVA.md mean. The REAL one is the project's submodules/containers (.gitmodules -> vasic-digital/Containers.git), which already has a complete, hardened, unit-tested pkg/egress (egress.go + egress_test.go + wave20_eg2hard_test.go, 12/12 tests independently re-verified passing this round) and scripts/egress-via-vpn.sh already exists and is already git-tracked (fail-loud on missing host real direct-egress-IP probing, honest failure when no tunnel exists). NEW THIS ROUND: test integration/test_egress_via_vpn.py – scripts/egress-via-vpn.sh previously had no dedicated test (only a bare bash -n cited in its own comments)

ATM ID	Type	Status	Severity	Description
				Adds 7 tests covering presence/exec-bit/syntax/usage/fail-loud-on-missing-host/fail-honest-on-dead-tunnel/real-direct-probe, plus the item's own exact acceptance criterion as an honestl @pytest.mark.skipif-gated test (SKIP: no VPN host configured, §11.4.3) – ready to run unmodified the moment BOBA_VPN_HOST is configured. Independently re-verified: 6 pass 1 honestly skipped. REMAINING WORK: (1) a operator-provisioned, SSH-reachable VPN host set via BOBA_VPN_HOST; (2) running the skip test (or scripts/egress-via-vpn.sh diagnose <known-blocked-tracker>) against it to produce the item's own exact acceptance evidence (via proxy egress IP != direct host IP AND a known blocked tracker returns 200 via proxy). No tracker is currently blocked from this sandbox host, so “known-blocked” is necessarily operator time-dependent and cannot be hardcoded. EVIDENCE: docs/qa/BOB-065/investigation_20260925.md.
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download proxy + qBitTorrent-go + Jackett + compose e forward
BOB-068	Bug	In progress	High	RD2-00: unattributed, unreviewed Auto-commit mechanism pushing to main
BOB-069	Task	In progress	High	RD2-40: §11.4.238 QA-discovery-channel ledger ongoing coverage-escape backfill
BOB-074	Task	In progress	Medium	RD2-07: DDoS-class testing fully absent from mandated test-type matrix
BOB-078	Task	Queued	High	RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit push script OR retire it
BOB-080	Task	Queued	High	RD2-13: Retroactive Fable-xhigh code review the substantive Auto-commit diffs
BOB-085	Task	Queued	Medium	RD2-18: Create top-level Boba proxy/merge-service v1.0.0 readiness ledger (closes GA-10)
BOB-090	Task	In progress	Medium	RD2-25: HelixQA Challenge entry exercising a three start.sh subcommands end-to-end against real compose stack

ATM ID	Type	Status	Severity	Description
BOB-093	Task	In progress	High	RD2-28: Live compose bring-up + verify rutra ReDoS regex bounds deployed to container (closes runtime half of GA-12)
BOB-094	Task	In progress	Medium	RD2-29: Author tests/stress/ test_tracker_fetch_stress_chaos.py with §11.4 fault injection
BOB-095	Task	In progress	Medium	RD2-30: Author tests/stress/ test_scheduler_hooks_sse_stress_chaos.py for side triangle
BOB-097	Task	In progress	Low	RD2-32: Author DDoS-class coverage for expo download-proxy/merge endpoints (canonical impl of RD2-07)
BOB-101	Task	Queued	High	OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): DEFER PAST v1.0 Go --profile go parity remains a goal but is LONGER a v1.0.0 release blocker. RW-10..13 n to a post-1.0 milestone and stop gating the tag Nothing advertised is removed, so §11.4.122 s clean and no §11.4.90 Obsolete closure is warranted. The BOB-141 measured fact stand unchanged: the Go container's Dockerfile run ONE binary binding only :7187; nothing bind :7186 or :7188 despite compose setting PROXY_PORT/BRIDGE_PORT. CLAUDE.md's po map already describes what the container ACTUALLY does and needs no amendment un this decision. This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inferen and supersedes any prior agent-chosen defau on this question. Options not chosen are nam above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item follows — GA-19/RW-09: Is --profile go parity a release goal? (gates RW-10..13) — OPERATO DECISION OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): KEEP 0.0.0.0 + MANDATORY AUTH GUARD The tunnel STA LAN-reachable — no bind address changes to
BOB-102	Task	In progress	Medium	

ATM ID	Type	Status	Severity	Description
				127.0.0.1. The operator explicitly chose to preserve access from other devices on the network. THE DECISION CARRIES A BINDING OBLIGATION: a permanent §11.4.135 regression guard MUST land that FAILS THE BUILD if an LAN-reachable route ever stops demanding authentication. The operator accepted 0.0.0.0 THE CONDITION that guard exists — without this decision is unprotected and the item is N closeable. Guard requirements: enumerate routes from the authoritative source (FastAPI app, Go handlers, boba-jackett) never a hand maintained list; resolve real auth coverage n grep for the string ‘auth’ (§11.4.201 real-condition); deliberately-public routes exempt ONLY via a checked-in list with per-entry justification, enumerated as honest gaps never silent; golden-TRUE + golden-FALSE fixtures per §11.4.107(10). This answer is recorded as consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item follows — RW-05: LAN-exposure threat mode bind tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION
BOB-104	Task	In progress	Medium	§11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge §11.4.238 followup: §11.4.84 quiescence-check helper for the unattributed auto-commit path === ROUND-2 PROGRESS, 2026-08-21 (discovered + independently verified 2026-09-25 via BOB- investigation) === Already substantially implemented at commit 7b45113c6b03524d9c799bdd68496ec202575e scripts/hooks/unattributed-commit-guard.sh (237 lines) + companion doc + a hermetic self-validated test (140 lines, golden-good/golden-self-test per §11.4.107(10)). The guard’s own
BOB-106	Task	In progress	Medium	

ATM ID	Type	Status	Severity	Description
				header carries an explicit, honest “WIRING (honest gap, §11.4.6)” section stating it is NOT invoked from scripts/pre_build_verification.s scripts/commit-push-all.sh. Confirmed via full diff grep the guard’s name appears only in its doc/its own test/BOB-162’s text – never inside either seam file. REMAINING WORK: (1) actually wiring into one of the two seams; (2) the §11.4.224(E) brownfield-adoption operator decision – 14 pre-existing violating commits would immediately fail the seam as written today, so wiring it in un-gated would retroactively “break the build” on history not is fixing. This item is explicitly tracked as blocked on that same operator decision via sibling item BOB-162 (“this item is not closed until that answer is recorded”). EVIDENCE: docs/qa/BOB-238/investigation_20260925.md (full citation trail).
BOB-109	Task	Ready for testing	Medium	BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix
BOB-111	Task	In progress	High	BOB-074 followup: configure real rate limiting for boba’s 3 public HTTP endpoints
BOB-121	Task	Ready for testing	Important	External watchdog for the forced-logout architectural gap (task #85, incident #3) Reported-Via: §11.4.202 reporting directive + on 2026-08-21T19:35:02Z Reported-By: BOB-107 BOB-107 remediation, residual What (the report, verbatim): The TESTS for both guards are now executed: pre-build invariant 30’s glob was extended from tests/unit + tests/pre_build also cover tests/hooks. MEASURED before/after the glob expanded to 27 suites with 0 under tests/hooks while 3 existed there; it now expands to 3 with 3 under tests/hooks. All three pass. But the GUARDS themselves are still invoked by nothing check-brief-inputs.sh is honestly conductor-rather than automatic: a brief’s required-input list lives in free-form prose that the Agent too structured input does not expose, so an automatic prompt-scraping gate would itself an unproven pattern-match. Its seam is the
BOB-162	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				dispatch procedure, and that is a documenta + habit change, not a hook. unattributed-comm guard.sh is different and is why this item existi Run against real history it names 14 violating commits since the last tag (and 21 bare Auto- commit subjects exist across all refs). Wiring into the pre-build or commit seam AS-IS wou therefore refuse every commit immediately, pre-existing debt nobody can fix in the mome That is precisely the 11.4.224(E) brownfield- adoption shape, and 11.4.66/11.4.122 put the choice with the operator, not with an agent inventing a ratchet. The options, stated so the decision is a decision and not a default: (a) immediate hard floor – the seam refuses until 14 are attributed; (b) one-time monotone- decrease ratchet (the 11.4.135 pattern) – snap 14 as the baseline, the count may fall and ma never rise, so day one is green and the diseas cannot spread; (c) changed-commits-only – g only commits created from now on, with a scheduled deadline for the historical 14; (d) report-only for a stated period, then escalate. Recommendation, with the reasoning rather just the pick: (b). It is the pattern this constitu already uses for exactly this situation, it mak the debt visible and bounded immediately, an cannot be satisfied by deleting the guard's na (11.4.227 closes that gaming channel). But it i operator's call and this item is not closed unt that answer is recorded. Honest boundary (11.4.6): this item does NOT claim the 14 com are harmful in themselves – it claims their ATTRIBUTION is absent and that nothing currently prevents the 15th. Affected scope / file-scope manifest: scripts/hooks/unattribu commit-guard.sh; scripts/hooks/check-brief- inputs.sh; scripts/pre_build_verification.sh; scripts/commit-push-all.sh Reproduction / context: Both guards run correctly by hand a are covered by passing tests (9/9 and 15/15, e proven non-bluff against a no-op stub). Neith invoked by scripts/pre_build_verification.sh c

ATM ID	Type	Status	Severity	Description
				scripts/commit-push-all.sh, so neither exerts standing detection pressure (11.4.226: a guard with no execution seam is not coverage; registration is not coverage). Acceptance criteria: Each guard is invoked by a named seam, OR carries a registered deferral pointing to this item. For unattributed-commit-guard.sh specifically, the operator has answered the adoption question below and the answer is recorded as consumer DATA – never an inverted ratchet. WHAT. The BOB-109 scaling suite gates its two timing tests on loadavg_1m/nproc <= 0.75 — on this 8-cpu host, a 1-minute load average at or below 6.0 — and SKIPS loudly otherwise. That gate is correct and was adopted for a good reason (§11.4.201(8)): the author validated a 2.10 growth-exponent threshold, then observed it to be FALSE-FAIL correct code at load 11.06 (baseline span 2.136), and at load 15-23 measured baseline spans of 2.358 against injected-cubic mutants 2.278-2.331 — the signal is smaller than the noise and the two are not separable in either direction. Gating was the honest response. THE PROBLEM. The gate has consequently NEVER RUN under its own precondition. Every recorded run today measured load 9.15-23; the independent reviewer measured 9.66 during review; the conductor measured 6.45 at its lowest. GREEN polarity WAS observed — at loads 9.4-11.6 (span 1.882) and in three stability runs (1.759/1.832/1.848) — but never once beneath the shipped ceiling. The author states this honestly as an inference from data rather than a run the reviewer can paste, and the reviewer confirmed the inference is sound (quiescence is strictly more favourable). Sound inference is still not an observed run. WHY THAT MATTERS (§11.4.226). A registered, topology-present guard that never executes is indistinguishable from a guard that would fail if it did. Nothing currently ensures it ever runs: no freshness contract, no scheduled
BOB-170	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				quiet-window invocation, no tracked item — which is precisely the unexecuted-standing-guard class §11.4.226 names, where registrat gets treated as coverage while execution is nobody's job. The forensic precedent in that anchor is two standing guards that, when fin executed, immediately emitted latent FAILs. ACCEPTANCE. (a) Execute both quiescence-ga timing tests on a host whose 1-minute loadav genuinely $\leq 0.75/\text{cpu}$ — no agent fleet, no concurrent build — and capture the run as a artifact showing the resolved load reading alongside the measured span exponent, so th precondition is provable from the artifact an not asserted. (b) Record the observed GREEN polarity in the item and in the test source, replacing the current honest-inference note. the gate FAILS under genuine quiescence, tha the finding this item exists to surface, and it reopens the threshold-calibration question rather than being worked around. (d) Establi freshness contract per §11.4.226: how stale a quiescent verdict may be before the gate cou as uncovered again, so this does not silently return to never-executing. HONEST BOUNDA This item does not claim the gate is wrong. Th evidence points the other way — quiescence strictly more favourable than the loads wher GREEN was already observed. It claims only the run has not happened, and that an unexecuted guard cannot be cited as coverag PROVENANCE. Raised as IMPORTANT-2 by th independent Fable-substrate reviewer of the BOB-109 work, and independently corroborate the reviewer verified the SKIP is loud and pri its resolved numbers, and confirmed the ship tests do SKIP at today's load. WHAT. The rutracker SEARCH endpoint is refused by bot protection while the site itself fully reachable. Measured unauthenticated 2026-08-21 (no cookie file read, no credential
BOB-172	Bug	Operator-blocked	High	

ATM ID	Type	Status	Severity	Description
BOB-184	Task	Queued	Medium	value in any artifact — §11.4.10): GET /forum/index.php -> 200, 96283 B, Three icon-glyph controls (.bridge-retry, .toggle, .caret) render text made of non-BMP code points. axe-core declines to decide their contrast, reporting them under the incomplete channel rule with messageKey nonBmp and resolving to NEITHER fgColor NOR bgColor, so the Python script docs/qa/BOB-164/axe_contrast_scan.py cannot decide either: 40 such nodes across 16 palette x mode combinations. BOB-164 round 2 stopped them from being scored as silent passes by naming them GLYPH_UNMEASURED, printing them on every run and fencing them, so any NEW unmeasured glyph node blocks. What remains genuinely unproven is their contrast itself: as non-text UI interface components they owe 3:1 under WCAG SC 1.4.11, and no oracle in this repo measures that today. Acceptance: a measurement path that resolves the effective foreground and background for glyph controls (for example reading getComputedStyle colour against the composed backdrop, or replacing the glyphs with inline text carrying explicit fill tokens), each of the three controls shown to clear 3:1 in all 16 palette x mode combinations, and the GLYPH_UNMEASURED fence shrunk by exactly the nodes then proven.
				docs/qa/BOB-164/axe_contrast_scan.py scans the static dist with no backend, so every node whose existence depends on fetched data is absent from the page it measures: the results table renders empty, so .type-badge and .quality-badge never appear, and the hooks list renders empty, so .status pills never appear. That is precisely why the BOB-164 round-1 regression went unseen in a rendered DOM even though it was scanned in all 16 themes, and it is why round-2 had to close the class in the arithmetic oracle instead. Those nodes are now covered by frontend/src/app/models/style-contrast.spec.ts
BOB-185	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
BOB-188	Bug	In progress	—	which extracts declared foreground-on-fill pairs from the stylesheets, but that is an ARITHMETIC claim about declared colour pairs, not a rendered-pixel one: it cannot see opacity applied by an ancestor, a composited backdrop, or a cascade this repo's static extractor does not model. Acceptance: the scan drives the dashboard with seeded fixture data (a stub backend, a route fixture, or an injected component harness) so the badge and status nodes are present in the DOM, axe measures them directly, and a control needle proves the rendered oracle sees a seeded sub-floor badge before any clean result from it is believed. OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): UNTRACK THE BINARY — BUILD ON DEMAND §11.4.30 and recorded as consumer DATA: the compiled workable-items binary is NOT to be git-tracked 'Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned' applies without exception here. Keeping it tracked with only invariant-52 fingerprint gate, and keeping it tracked with a commit-time rebuild, were both offered and NOT chosen. REQUIRED END STATE the gate builds from source, or REFUSES HONESTLY when the Go toolchain is absent (§11.4.201(11) — probe the artifact through its real invocation path, never fake a pass, never silently fall back to a stale binary). Staleness becomes structurally impossible rather than merely detected. ACCEPTED COST: a fresh clone needs a Go toolchain before invariant 17 can be — already true for the qBitTorrent-go backend so this adds no new host requirement. The invariant-52 fingerprint gate remains useful defence-in-depth for as long as any binary exists on disk. This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question.

ATM ID	Type	Status	Severity	Description
				Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item text follows — pre_build_verification.sh invariant 17 (CM-WORKABLE-ITEMS-VALIDATE) resolves its binary through the candidate loop at :534, where the FIRST entry is constitution/scripts/workable-items/bin/workable-items. That file is GIT-TRACKED (md5 17644a248363, identical to bin/workable-items-linux) and executable, so it VETOES resolution over the current untracked sibling constitution/scripts/workable-items/workable-items (md5 43376a6d0184). The tracked binary is STALE: it does not contain the guards its own source now has. MEASURED, needle-proven, 2026-08-25. String presence in the tracked binary: “refusing to set terminal status” -> 0; “Issues-location item has TERMINAL status” -> 0; compare needle from the SAME updateCmd, “at least one mutable field flag is required” -> 2 (so the instrument sees through that path and the zero are real absences, not a blind read). The untracked sibling returns 1 / 1 / 2 for the same three queries. A check whose message string is absent from the binary cannot fire. RUNTIME PROOF: the same injected violation (a row set terminal status while current_location=Issue) run through both binaries -> stale reports 1 violation (only the body_md desync class), current reports 2 including the location-status class verbatim. This is exactly why the ten BOB-166 rows (BOB-087/129/131/144/145/146/148/153/155/156) survived undetected: the gate that was supposed to catch them was running a binary structure incapable of seeing them. The 11.4.108 SOURCE >ARTIFACT gap: source correct, artifact stale, every gate reading the artifact reports green. This is the THIRD instance of that class found in one session, alongside BOB-169 (a charset gate that never opened a file) and BOB-183 (a service bundle five days older than its sources).

ATM ID	Type	Status	Severity	Description
BOB-191	Bug	Queued	—	COMPOUNDING: the comment at :519-523 directly above the loop asserts “The naive bin/workable-items path never existed in this checkout (bin/ is a gitignored local build-outp dir nothing ever populated)”. That statement is now FALSE - bin/ holds two tracked binaries. The stale comment asserting the absence of the v file that now shadows resolution is how this stayed invisible. Also an 11.4.30 question the operator owns: bin/workable-items and bin/workable-items-linux are VERSIONED BUILD ARTIFACTS in a constitution submodule that is inherited by reference by every consumer, so every consumer inherits whichever binary was last committed. Flagged independently by two separate agents this session. ACCEPTANCE: (a) resolution must never prefer a stale artifact over a current one - either the tracked binaries are removed and resolution falls through to build on-demand, or a freshness check refuses a binary older than its sources (see the BOB-18 fingerprint gate for a working pattern); (b) the false comment at :519-523 corrected; (c) a pair 1.1 mutation proving the new refusal fires, plus negative control proving a genuinely fresh binary still passes so the fix is not an 11.4.201 false-positive refusal; (d) the 11.4.30 tracked-binary decision recorded as operator DATA. Discovered out-of-band during BOB-166 remediation - a 11.4.238 coverage escape in its own right. WHAT: the §11.4.252 fail-open scan reports 0 for qBitTorrent-go, and that zero is NOT evidence. The triage agent ran control needles through the scanner’s own path per §11.4.201 (b): a Python ‘except Exception: pass’ needle WAS SEEN, a TypeScript ‘catch (e) {}’ needle was SEEN (so frontend/src’s zero IS a real zero), but a G empty-‘if err != nil {}’ needle was NOT SEEN. The instrument that cannot see the idiom returns the same quiet zero as a clean tree, and only one of those is honest. DISTINCT FROM BOB-189,

ATM ID	Type	Status	Severity	Description
				deliberately not merged with it per §11.4.214 BOB-189 is a false MATCH (fail-closed guards reported as fail-open); this is a false NULL (an entire language unscanned). Same scanner, opposite failure directions, different fixes - merging them would hide one behind the other IMPACT: Go is the language of qbittorrent-pr... go and boba-jackett (port 7189, which owns encrypted tracker credentials), so the unscanned surface is exactly where §11.4.252's credentials plus-mutation combination is most likely. Nobody has assessed it; the dashboard says clean. ACCEPTANCE: the scanner grows a Go covering the empty-err-block and swallowed error idioms, its needle is SEEN through the r... path, qBitTorrent-go's result is re-derived, and every finding is triaged as this Python/TS pass was. Until then qBitTorrent-go's fail-open position is UNKNOWN and must be reported as UNKNOWN, never as 0. == ROUND-2 INVESTIGATION, MEASURED 2026-08-26 — T FILED TITLE WAS TOO NARROW (§11.4.6) == The live scanner is constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh, driven per-root by invariant 39 at scripts/pre_build_verification.sh:1481-1544. (scripts/pre_build/check_cm_no_fail_open_skip.sh has NEVER existed on any branch — it exists only in an uncommitted worktree. Any citation of the path is citing a file that is not there.) IT IS A WEAK-MATCHER DEFECT, NOT A SCOPE HOLE Line :243 sets exts="py go rs c cc cpp h hpp ja cs js ts jsx tsx php rb" — .go IS enumerated. B... line :537 gates the Python ast analyser on cases "\$f" in *.py), so every non-Python file falls through to just two text matchers (:772 empty catch (...) {, :794 credential = x \\\ "literal"). Go has no catch keyword, so neither matcher CAN fire on Go by construction. NEED TEST, BOTH ARMS (planted / rc / result): Python 2 / 1 / SEEN · TypeScript 1 / 1 / SEEN · Java 1 / 1 / SEEN · Go 6 / 0 / NOT SEEN — PASS · Rust 2 / 0 / NOT SEEN · Ruby 1 / 0 / NOT SEEN · C 2 / 0 / NOT

ATM ID	Type	Status	Severity	Description
				SEEN. Go is ONE OF FIVE BLIND EXTENSION not a special case. The Go needle is real code. build rc=0, go vet rc=0, gofmt -e clean. THE DECISIVE DISCRIMINATOR — same bytes, extension changed: .zig (NOT enumerated) → “SKIP — topology_unsupported”; .go (enumerated) → “PASS”. The gate ALREADY OWNS an honest-blindness mechanism and it works correctly; Go routes around it precisely BEING ENUMERATED. A scope hole announces itself; this matcher hole prints green. That makes it the worse defect and is why the corrected tree leads with it. QUANTIFIED SURFACE (exclusion stated, §11.4.224(E)): using the scanner’s OWNED prune list and nothing added — 106 .go files / 20,389 LOC (52 production files / 6,858 LOC). WITH and WITHOUT exclusions the count is == 106; no vendored tree exists, so no unstateful exclusion could manufacture a clean number. qBitTorrent-go holds 106 blind files and 0 analysable files — 100% blind — yet invariably counts it among “5 first-party source root(s)” reported clean. DEFECTS THE BLINDNESS HOLE filed as BOB-204 (credential DELETE returns unconditional 204 while the .env plaintext de error is discarded; plus the env_write_failed_db_rolled_back code asserts an unconfirmed rollback). SEPARATE SCOPE HOLE → filed as BOB-205 (cmd/boba-ctl, 947 LOC orchestrator, absent from DANGER_ROOTS entirely). Distinct-but-similar per §11.4.214, deliberately not merged. GOLDEN-FALSE FIXTURE FOR ANY FUTURE GO ARM: internal/jackettapi/auth_middleware.go:45-66 is correct fail-CLOSED (constant-time compare, 401+ret — a Go arm MUST NOT flag it. LOAD-BEARING §11.4.250 PREDICTION: bolting a naive Go re arm on will IMMEDIATELY manufacture BOB-189-class false positives in Go — if err nil { return false } in a Go validator is fail-CLOSED. Fixing this as layer N+1 reproduces BOB-189 in a new language. THEREFORE the remediation order is fixed: (1) analyser

ATM ID	Type	Status	Severity	Description
				REGISTRY with an UNANALYSED verdict FIRST extension→analyser map, unmapped extensions reports UNANALYSED never silent PASS, which converts the false null into an honest gap and covers rust/ruby/C too; (2) only then a Go arm call-site-aware from day one (go/ast, or adopted errcheck/staticcheck rather than hand-rolled regex), shipped with the auth_middleware golden-FALSE. ONE PRIMITIVE, TWO SYMPTOMS: the scanner classifies by local syntactic shape and never consults semantic — BOB-189 ignores the CALLER's context (false match), BOB-191 searches for a shape from a DIFFERENT language family (false null). The gate's own header already concedes the primitive ("requires real data-flow analysis this gate CANNOT honestly claim"); what is missing is propagating that concession into PER-LANGUAGE honesty. Keep two tracker items. INSTRUMENTED BUG RECORDED, NOT HIDDEN (§11.4.201(12)) the investigation's first verdict extraction used <code>grep -oE 'PASS\ FAIL\ SKIP'</code>, which matched FAIL inside the gate's own name (...FAIL-CLOCK) and mislabelled three PASSes. Re-run keyed on exit code; the table above is the corrected run EVIDENCE: docs/qa/BOB-191/scanner_blindness_investigation_20260826.m (255 lines, §11.4.44 header). No source, gate, or config modified by the investigation. === ROUND-3 PARTIAL FIX, MEASURED 2026-09-2 STEP (1) OF THE ROUND-2 REMEDIATION ORDER LANDED; STEP (2) STILL OPEN, ACCEPTANCE NOT YET MET === Per the round-2 remediation order, step (1) — the analyser REGISTRY with honest UNANALYSED verdict — is now implemented and independently re-verified (conductor re-ran the mutation suite from a clean shell, 206/206 green). Step (2) — a real semantic Go arm — is DELIBERATELY DEFERRED, so this item stays Queued: the ACCEPTANCE criterion above ("its needle is S through the real path") is still unmet for Go/Ruby/C. CORRECTION TO ROUND-2's OWN

ATM ID	Type	Status	Severity	Description
				FIGURE (§11.4.6, re-verified live, not re-guess round-2 said “5 blind extensions (go/rs/rb/c/h Re-run live against all candidates this round: (.cpp, a genuine empty catch() {}) is SEEN — rc=1 FAIL, already correctly caught. It was ne actually blind; .h is content-ambiguous (same header extension serves both C and C++) and not a distinct 5th case. Exactly 4 languages are structurally blind: Go, Rust, Ruby, C. This narrows, not widens, the round-2 figure. WHEN LANDED (constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh — constitution-submodule file, inherited by reference; landing it for real still needs the submodule’s own §11.4.26 fetch/pull/push workflow, separate from boba’s commit path. New DANGEROUS_COMBO_UNANALYSED_EXT registry (default go rs rb c). Verdict now has 3 branches: real hits → FAIL (unchanged); zero hits + UNANALYSED files present → new honest Δ M — N file(s) UNANALYSED... + PASS (PARTIAL COVERAGE) (still exit 0, advisory — never block build on an unanalysed language); zero hits - zero unanalysed → unchanged original clean PASS text. - scripts/pre_build_verification.invariant 39 (boba-owned): fixed to read the UNANALYSED count unconditionally, not only nonzero exit — otherwise a 100%-UNANALYSED root would still have silently printed “no fail-open anti-pattern across N clean roots” with caller never seeing the NOTE. - Mutation test extended with 17 new assertions covering Go/Rust/Ruby/C UNANALYSED-branch RED/GREEN C++ negative control (still caught, never marked UNANALYSED — the registry must not swallow working extension), a clean-root negative control, and a mixed-real-hit-not-masked control. Full suite 206/206 (189 pre-existing + 17 new) independently re-run by the conductor from clean shell. LIVE FULL-REPO RESCAN RESULTS (real DANGER_ROOTS, this round): Python backlog unchanged at 59 hits (download-prod:src:25, plugins:32, scripts:2 — pre-existing, ou

ATM ID	Type	Status	Severity	Description
BOB-194	Bug	In progress	—	scope, advisory since 2026-08-20). New honest signal: 121 files now reported UNANALYSED (qBitTorrent-go:117, cmd/boba-ctl:4 — BOB-2026-08-20). The scope hole now visibly folded into the same honest-gap mechanism) instead of silently counting toward “clean.” Zero new FAIL findings in Go — by design; a real Go checker needs context-aware semantic analysis, still deferred. VULNERABILITY STEP (2) STAYS DEFERRED, NOT A REGRESSION. The round-2 §11.4.250 PREDICTION stands unrefuted — a naive Go regex arm would immediately flag internal/jackettapi/auth_middleware.go:45-66 (the golden-FALSE fixture) as a BOB-189-class false positive. A real arm needs go/ast or an adopted tool (errcheck/staticcheck), shipped WITH that golden-FALSE passing, before it can fire. REMAINING WORK TO CLOSE THIS ITEM: implement the Go semantic arm (go/ast-based or errcheck/staticcheck-adopted) covering the empty-err-block and swallowed-error idioms, verified against the auth_middleware.go golden-FALSE fixture, re-derive qBitTorrent-go’s real result (currently UNKNOWN, honestly reported as such), and triage every real finding as the Python/TS pass already is. Rust/Ruby/C have zero first-party findings in boba’s tree today (verified this round) so their own arms are lower priority until such files emerge — the UNANALYSED registry already covers them honestly in the meantime. EVIDENCE: cmd/qa/BOB-191/closure_evidence_20260925.md (RED/GREEN transcripts, the C++ correction with live proof, the live full-repo scan table, anti-bug provenance). State left uncommitted by design (constitution-submodule files need the §11.4.250 workflow, not boba’s commit-push-all.sh). WHAT: agent worktrees under .claude/worktrees/ are ephemeral copies pinned at arbitrary commits - never built, never shipped, deleted when the agent finishes. Pre-build guards that walk the tree with find(1) from the repo root descend into them anyway. MEASURED

ATM ID	Type	Status	Severity	Description
				INSTANCE (2026-08-25): CM-GO-TOOLCHAIN-MATCHES-BUILDER FAILED the main build w exactly one finding, from .claude/worktrees/agent-afda1df906c537ab4 - a worktree 86 commits behind still carrying 'FROM golang:alpine' against a go.mod long since at 1.26.2. 'main tree was CORRECT throughout. Per §11.4.201(1) a false-positive refusal is a FAIL- of equal severity to a false pass: it blocks real work and teaches people to bypass the gate. FIXED FOR ONE GATE: '.claude' added to that gate's PRUNE_DIRS with a deterministic §1.1 - a stale mismatch inside .claude/worktrees n NOT fail a healthy tree, the SAME mismatch outside it MUST still fail, so the prune cannot widen into blindness (§11.4.201(6)). Mutation verified: reverting the prune fails 2 checks. SURVEY, measured not inferred: 8 of 13 pre_b gates walk the tree; only the fixed one prunes .claude. Gates using 'git ls-files' are structurally immune - worktrees are untrack Empirically probed: check_cm_closure_seam_binds, check_cm_killpg_pgid_guard and check_cm_no_production_mutation_residue rc=0 with ZERO worktree references - clean i practice today. UNMEASURED, not clean: check_cm_test_mock_pid_explicit_int and check_cm_test_mock_pid_patched_when_rea were not reached before the probe budget expired. HYPOTHESIS REFUTED, recorded so nobody re-walks it: an earlier revision of this item speculated that check_cm_plugin_count 90s timeout was caused by walking worktree is NOT. That gate's find is scoped to \$PLUGINS_DIR (check_cm_plugin_count.sh:2 so worktrees are outside its walk entirely. Its slowness is real - still running past 54s on a r measure - but has a different, still-unidentifie cause and belongs to its own item, not this or ACCEPTANCE: every tree-walking gate either prunes agent scratch or is proven immune by measurement, each with a paired mutation.

ATM ID	Type	Status	Severity	Description
BOB-211	Bug	Queued	minor	NOTE the asymmetry that bounds the risk: a worktree can only ADD findings to a security relevant scan (killpg, mutation-residue), never remove them - so the exposure is false refusal and eroded trust, not a missed defect. STATUS: LATENT on this host (measured: all declared locations sit on ONE btrfs mount, full ownership-expressive) and OPERATOR-GATED for verification. Retained from the dismissed BOB-206 rather than discarded with it, because the scenario is well-founded elsewhere: the declared mount is a udisks auto-mounted REMOVABLE NVMe, so another operator's portable drive being exFAT or NTFS is entirely ordinary. TWO RESIDUALS: (1) REPAIR-LAYER DIAGNOSIS. On a uid-flattening mount whose flattened uid is NOT the operator, probe_location correctly REFUSES (verified: uid=0 and uid=100999 both refuse). But it then hands ownership_repair.sh a chown that will return EPERM, and nothing in the repair path is fstyp aware — so the operator sees a permission failure whose real remedy is a REMOUNT OPTION, which no message says. Reasoned, not proven: the shim used to verify BOB-206 mode the stat READ, not the chown WRITE, and cannot settle whether chown actually fails. (2) MODE CONTRACT. vfat flattens MODE via fmask/dm so the preserve_mode: true / 600 contract on and config/boba.db would be silently unenforceable there. §11.4.10-adjacent: a credential file believed to be 600 that is world-readable by mount option is a leak the mode check cannot see. WHY OPERATOR-GATED: a genuine end-to-end RED needs a real uid-flattening mount. The privilege-free route is bindfs -u (FUSE; fusermount IS present and user_allow_other IS set) — but bindfs is ABSENT on this host, as are mkfs.vfat and losetup. So verification needs either a bindfs install or a privileged loopback mount. Neither was attempted (§11.4.21 — high-blast-radius host

ATM ID	Type	Status	Severity	Description
BOB-219	Bug	In progress	major	mutation is operator-gated). ALSO RECORDED NO test anywhere exercises a uid-flattening filesystem — vfat/exfat/ntfs/flatten/loopback/mkfs/losetup/bindfs/fusermount all measure across the three ownership test files, needle-proven (probe_location = 3 hits through the s instrument). ACCEPTANCE: (1) operator decide whether to install bindfs or provide a privilege loopback so the RED can be built; (2) if verified the repair path gains fstype-aware diagnosis naming the remount remedy; (3) the mode contract either detects mode-flattening mount and refuses, or documents honestly that it can be enforced there. DISCOVERY CHANNEL (§11.4.238): retained residual from the BOB-2 verification stream. WHAT, conductor-verified: docs/guides/tracker-credentials.md is TRACKED. Its §11.4.65-mandated export twins docs/guides/tracker-credentials.html and .pdf both EXIST ON DISK are NOT tracked, and ARE ignored — matched the .gitignore <i>credentials</i> deny-all at :31. The . is rescued by an explicit allowlist entry at :38 nobody added entries for the twins. WHY IT IS A LIVE VIOLATION AND NOT A HYPOTHETICAL: §11.4.65 requires every in-scope Markdown document to carry synchronized .html/.pdf siblings, and §11.4.212 requires every §11.4.65-scope doc to be reachable from the README. Two artifacts that exist locally but can never be committed are, from any fresh clone, ABSENT so every clone of this repository has a tracker-credentials guide with no exports, while the authoring host looks complete. The divergence is invisible on the machine that would notice it. WHY NOBODY SAW IT: this is a §11.4.201(6) FALSE-NULL of the BOB-212 class. git status never listed the twins, so no gate and no author had a signal. The blind instrument and the clone tree return the identical quiet zero. ANOTHER INSTANCE OF THE SAME MECHANISM, filed separately rather than separately because it is one fix: d

ATM ID	Type	Status	Severity	Description
				qa/BOB-124/evidence/loginctl_user_state.txt is swallowed by *_user* at .gitignore:~55 and is therefore uncommitted §11.4.83 QA evidence SIBLINGS IN THE SAME DIRECTORY — user1000_grepped.log, user_scope_events_head.log, user_service_lifecycle.log — ARE tracked, saved only by the robust glob negation !docs/qa/**/. That one file is lost purely because it is .txt rather than .log. The pattern is exact: robust glob negations hold; per-file and per-extension rescues leak. ACCEPTANCE: (1) both twins become trackable and are committed alongside their .md; (2) the BOB-124 .txt evidence likewise; (3) whichever BOB-212 fix direction is chosen MUST cover these — a fix that closes the future to swallow while leaving these three artifacts permanently uncommittable has addressed the mechanism and not the damage; (4) a check to ensure a tracked .md in §11.4.65 scope has COMMITTABLE twins, so this class cannot regress silently. DISCOVERY CHANNEL (§11.4.238): found by the BOB-212 blast-radius sweep — and only on its SECOND pass. The first pass filtered by a source-extension set that omitted .pdf, which hid this finding entirely; the agent re-ran with no extension filter over all 420 non-artifact paths and recorded the blind spot rather than shipping the first number. Worth keeping: an extension allowlist is itself a false-null generator, which has the same shape as the defect being investigated. === ROUND-2 PARTIAL CLOSURE, MEASURED 2026-09-25 — criterion 1 already done, criterion 2 unsatisfiable, criteria 3+4 done ONLY for the .md-export-twin class === Independently verified via git check-ignore -v on all three named artifacts: CRITERION 1 (tracker-credentials.{html,pdf} twins committable) — ALREADY DONE, pre-dating this round: commit 9f6f517 already added !docs/guides/tracker-credentials.{html,pdf} rescue lines (.gitignore:49-54); both are tracked and clean. Nothing to do. CRITERION 2 (BOB-124

ATM ID	Type	Status	Severity	Description
				loginctl_user_state.txt trackable) — CLOSED A UNSATISFIABLE. The file does not exist on disk and has ZERO trace in <pre>git log --all --diff-filter=A --</pre> "*loginctl_user_state*", and is absent from the entire working tree. Its .log siblings in the same directory ARE tracked (rescued by the existing docs/qa/**/*.*.log negation), confirming the file was lost purely by having a .txt extension, exactly as the original report claimed — but there is no recovery path and no legitimate way to “make trackable” without fabricating QA evidence, which is itself an anti-bluff violation. This criterion is permanently closed, not deferred. CRITERION 3 (general mechanism preventing recurrence) — PARTIALLY closed. A new guard (see criterion 4) closes the class criterion 1 exemplified: a TRACKED .md file whose §11.4 export twin is gitignore-swallowed. It does NOT close the class criterion 2 exemplified: a NON-.md QA-evidence file (any extension) swallowed by an unrelated gitignore pattern. Extending the existing !docs/qa/**/*.*.log negation to also cover .txt was concretely measured this round and PROVEN UNSAFE for this project: .gitignore documents cookies_* as this repo’s own secret-file naming convention and a scratch-tree experiment confirmed a .txt-wide rescue would also un-ignore cookies_rutacker.txt / RUTACKER_session.txt my_password.txt — real credential-shaped files already relying on that exact pattern staying broad. This candidate fix is now closed off with evidence rather than left for a future session re-attempt and re-measure. THE REMAINING GAP: a general mechanism for “a QA-evidence file legitimately meant to be tracked gets silently swallowed by a broad secret-shaped-filename gitignore pattern, distinguished from an actual secret file” remains genuinely open — it needs a smarter signal than file extension (e.g. content sniffing, or an explicit per-file allowlist-on-creation discipline) that this round did not

ATM ID	Type	Status	Severity	Description
BOB-231	Task	Queued	Major	attempt to design, since inventing one hastily risked exactly the credential-leak class the constraint exists to prevent. CRITERION 4 (a check exists) — DONE for the .md-export-twins class. New gate scripts/pre_build/check_md_export_twins_committable.sh + self-validated test tests/pre_build/test_check_md_export_twins_committable.sh arms: RED, GREEN, 2 golden-FALSE, fail-close x3, real-repo run, §1.1 paired mutation — all independently re-run and confirmed passing the conductor). Wired into scripts/pre_build_verification.sh as invariant 58/58 (MD-EXPORT-TWINS-COMMITTABLE), mirror the sibling CM-GITIGNORE-SWALLOW-GUARD (BOB-212) pattern exactly — BLOCKING, not advisory, since it is a pure tree-scan needing live stack. REMAINING WORK TO FULLY CLOSE THIS ITEM: design and implement the criterion residual — a mechanism distinguishing “a legitimate QA-evidence file lost to a secret-sh gitignore pattern” from “an actual secret correctly caught by that pattern” — without narrowing the existing credential-protecting globs. Until then this item stays Queued; criteria 1/2/4 are genuinely done, criterion 3 is genuinely partial. EVIDENCE: docs/qa/BOB-219/closure_evidence_20260925.md (full command transcripts: before/after git check-ignore, the wide-rescue safety experiment, 7-arm test output, standalone gate invocation against the real repo). BOB-227 measured 3 of 4 artifacts of the CM-LAN-ROUTES-AUTHENTICATED gate untracked; git; criterion 1 (commit them) was already resolved by an unrelated prior commit before this session, closed 2026-09-23. Criterion 2 was NOT addressed: ‘a gate asserting that every executable a pre-build invariant invokes is tracked’ – a general mechanism preventing the whole CLASS of defect (a pre-build gate’s own implementation files shipping untracked,

ATM ID	Type	Status	Severity	Description
				invisible to a fresh clone, no committed base for round-over-round diffs) from recurring for ANY future gate, not merely this one. ACCEPTANCE: (1) enumerate every file path scripts/pre_build_verification.sh invokes (via bash/timeout/python3 calls to scripts under scripts/pre_build/, plus every tests/pre_build/ and tests/hooks/.sh it runs) – likely via a static grep/parse of pre_build_verification.sh itself, runtime trace; (2) assert every one of those p is git-tracked (git ls-files –error-unmatch); (3) wire this as a new pre-build invariant so a fu untracked gate implementation is caught immediately, not discovered independently weeks later; (4) a RED test creating an untrac fake gate-invocation target and asserting the check fails on it, GREEN once the mechanism exists and the fake target is either removed o tracked. Live kinozal credential test reports authenticated=False, status='empty', error=' (found 2026-09-23). ROOT CAUSES (docs/qa/ BOB-235/investigation_20260923.md): (A) operator/environment: kinozal.tv publishes A record 127.0.0.1 (host, container, DoH Cloudflare+Google all agree; rutracker.org resolves normally as control), KINOZAL_MIRRORS is empty so search.py fal back to kinozal.tv; the live mirror kinozal.gu answers every non-browser client with a Cloudflare 403 'Just a moment' challenge (log POST, curl, aiohttp, operator cookies with two user-agents; no cf_clearance cookie; KINOZAL_COOKIES is loaded into the contain but no code reads it). (B) code defect FIXED under TDD (RED 2 failed/1 passed, GREEN 19 passed): _search_kinozal's except branch only logged and stashed no diagnostic, so the chip read empty with error=None; it now stashes error_type + 'Kinozal request failed: '. Needs start.sh –reload-python to be live. Credential validity is UNKNOWN (no request ever reach
BOB-235	Bug	Operator-blocked	—	

ATM ID	Type	Status	Severity	Description
BOB-237	Bug	Queued	high	login endpoint) - do not rotate credentials on evidence. Operator-Block-Details: WHAT: cho (1) reachable domain - set KINOZAL_MIRRORS=https://kinozal.guru + ./start.sh –recreate, or change the roster prima in trackers.py; (2) how to pass the Cloudflare challenge - FlareSolvrr (not deployed), cf_clearance cookie + code that actually reads KINOZAL_COOKIES, Jackett’s kinozal indexer, mark kinozal operator-blocked; (3) optionally export a kinozal-only cookie file. WHY: every agent-reachable path was exhausted - DNS, mirrors, login POST, cookies, env propagation measured. UNBLOCK CONDITION: the live test authenticates or the operator marks kinozal unsupported. WHO: repository operator. WHAT: CM-GATE-LEDGER-RATCHET (§11.4.227(A)) FAILED the pre-build sweep during this session’s batch-15 commit attempt: unimplemented=414 exceeds the checked-in baseline=403 in constitution/scripts/gates/gate_ledger_baseline.txt. This is a HARD, block FAIL (unlike the other gate findings surfaced the same sweep, which are all explicitly ADVISORY/non-blocking per §11.4.234). INVESTIGATION (§11.4.102): the four newest anchors this session’s constitution pull brought in (§11.4.268 tamper-evident evidence chain, §11.4.269 critic/consensus advisory-only ban, §11.4.270 dependency-existence-verdict register §11.4.271 waiver mechanism) were checked FIRST as the likely cause, since they are the most recently landed (2026-08-26) and each names several recommended mechanism gates in its own text. RULED OUT: every one of those gate names (CM-CRITIC-CONSENSUS-ADVISORY-ONLY, CM-DEPENDENCY-EXISTENCE-VERDICT-REGISTER, CM-EVIDENCE-CHAIN-ANCHOR-CATCHES-RECOMPUTE-AND-TRUNCATION, CM-EVIDENCE-CHAIN-DELETE-REORDER-DETECT, CM-EVIDENCE-CHAIN-INCOMPLETE-VERIFICATION-REFUSES, CM-WAIVER-

ATM ID	Type	Status	Severity	Description
				ROSTERED-EXPIRY-TRACKED) is already correctly registered DEFERRED against an OWED-GATE-NNN tracked item, and each anchor's own §11.4.227(B) propagation gate is IMPLEMENTED. So this batch of new anchors will NOT introduce the ratchet violation. The full ledger run (docs/qa/BOB-237/unimplemented_gates_snapshot_20260925.tx) is 414 lines, captured verbatim from bash constitution/scripts/gates/cm_gate_ledger_ratchet.sh) shows the total current state is 414 UNIMPLEMENTED + 121 IMPLEMENTED + 73 DEFERRED. The checked-out baseline (403) was last legitimately bumped by commit 34e42f0 ("cite the §11.4.209 gate rename bump baseline 397->403") — some point after that, the unimplemented count grew to 414 (403 + 11 more) without a matching baseline bump or deferral registration for the newly-named gates across one or more subsequent anchor-landing rounds in the constitution submodule. This is accumulated UNIVERSAL governance-corpus debt, not something introduced by any boba-gate edit in this session (confirmed: git diff eba38e8..HEAD -- constitution/scripts/gate_ledger_baseline.txt inside the constitution submodule is empty — the baseline file itself never touched across the two merges this session performed). WHY DEFERRED RATHER THAN FIXED HERE: identifying which of the 414 unimplemented names are the specific 11 that pushed past baseline, and then either implementing 11 gates' worth of real UNIVERSAL gate-code or registering 11 proper OWED-GATE-NNN deferrals with tracked-item citations, is itself real engineering work scoped to the constitution submodule (universal, not project-specific per §11.4.17) — it is not a boba-project and doing it hastily mid-session, unrelated to session's actual mandate (Jackett incorporation backlog triage), risks exactly the kind of rushed unreviewed governance-corpus edit §11.4.209 Fable-xhigh review requirement exists to

ATM ID	Type	Status	Severity	Description
				prevent. ACCEPTANCE: (1) a future session (on the constitution's own maintainers) triages the 414-line snapshot against the 403 baseline to identify the specific unaccounted gate names; each is either implemented, registered as a proper OWED-GATE-NNN deferral, or (if genuinely retired) cited in constitution/scripts/gates/gate_ledger_removals.tsv; (3) the constitution submodule's baseline is re-bumped with a citation once accounted, following the same pattern as commit 34e42f0; (4) boba's pinned constitution pointer is then re-synced; this item closed with the passing CM-GATE-LEDGER-RATCHET re-run as evidence. DISPOSITION FOR THIS SESSION'S COMMIT: §11.4.234(D) (the commit/push mechanism MUST always be able to complete; a failing gate is skippable ONLY via an explicit recorded deferral flag, with the skip recorded in the commit message so the debt stays tracked, never forgotten), this session's batch-15 commit uses the sanctioned skip (BOBA_SYNC_SKIP_CI=1), citing this tracked item BOB-237, rather than silently bypassing or spending unbounded effort resolving an inherited universal-corpus debt mid-session. DISCOVERY CHANNEL (§11.4.23) found by boba's own pre-build sweep (scripts/pre_build_verification.sh, invariant 38/57) during a routine session commit — the automated gate itself is the discoverer, exactly as §11.4.238 requires. BOB-223 closed the incentive inversion when writing a §11.4.18-mandated docs/scripts/mods/companion doc tripped invariant 16 while the mandate requiring the doc to exist (CM-SCRIPTS-DOCS-SYNC) has zero implementation, and the doc's required §11.4.44 revision header (CM-SCRIPTS-REVISION-HEADER-PRESENT) is likewise unenforced outside one unrelated ledger file. Both are now registered as explicit GATE-DEFERRAL REGISTER deferrals in scripts/pre_build_verification.sh, satisfying
BOB-242	Task	Queued	—	

ATM ID	Type	Status	Severity	Description
BOB-243	Bug	Queued	—	§11.4.227(A)’s bar for now, but the underlying mandates remain unenforced. This item implements BOTH: (1) walk every <code>.sh/.bash u scripts/</code>, require a docs/scripts/.md companion (24 of 36 currently lack one per BOB-151’s own count), verify same-commit-or-newer mtime §11.4.18’s literal text; (2) require every in-scope companion doc to carry Revision:/Last modified: per §11.4.44. Both need an operator brownfield-adoption decision (§11.4.224(E)/ §11.4.66 – immediate hard floor vs. monotonic decrease ratchet vs. changed-files-only), a pair §1.1 mutation each, and – once landed – removal of both DEFERRED: lines from the GATE-DEB REGISTER in scripts/pre_build_verification.sh Duplicate-of/originating-from: BOB-223, BOB-212. Also recommend adding two rows to constitution/scripts/gates/gate_ledger_deferrals.tsv (the constitution submodule’s own canonical §11.4.227(A) deferral ledger) once this new item’s ID is known. Discovered by a full 58-invariant pre_build_verification.sh sweep (2026-09-25) during BOB-223 closure verification: invariant (CM-GITIGNORE-SWALLOW-GUARD, §11.4.20 BOB-212) reports 4 first-party source files silently ignored: <code>.specify/extensions/superspec/scripts/e2e-agent-claude.sh</code>, <code>.specify/extensions/superspec/scripts/e2e-smoke.sh</code>, <code>.specify/extensions/superspec/scripts/validate-extensions-metadata.py</code>, <code>.specify/extensions/superspec/scripts/validate-release-archive.py</code> – all blocked by the broad <code>.gitignore:218</code> rule <code>‘.specify/extensions/superspec/’</code>. Unrelated to BOB-223’s own scope; a genuinely new finding, not previously tracked. Remedy per the gate’s own remediation text: either rename/relocate them so no <code>.gitignore</code> rule matches them, or add an explicit <code>!</code> negation for them in <code>.gitignore</code>, then run the guard. Investigate first per §11.4.6/ §11.4.124 whether the broad <code>.specify/extensions/superspec/</code> ignore rule was intentional (e.g. to

ATM ID	Type	Status	Severity	Description
				exclude a vendored/generated subtree) before narrowing it, since a careless negation could expose something the rule was deliberately protecting. Discovered 2026-09-25 by the pre-build sweeper on own CM-GITIGNORE-SWALLOW-GUARD gate (§11.4.201(6), BOB-212 pattern), during the 00-zero-shortcomings-audit feature’s setup phase. What: 4 genuine first-party files under .specifications/extensions/superspec/scripts/ — e2e-agent-claude.sh (19458 bytes), e2e-smoke.sh (8757 bytes), validate-extension-metadata.py (5888 bytes), validate-release-archive.py (7053 bytes) — all dated Aug 31 2026 — are untracked by git (confirmed via ‘git ls-files –error-unmatch’: does not match any file(s) known to git) and are BLOCKED from ever being staged by the broad directory-level ignore rule at .gitignore:218 (.specifications/extensions/superspec/). Root cause investigation needed: .gitignore:218’s own comment states the WHOLE directory is ignored because it is ‘the vendored superspec extension checkout’ that ‘carries its OWN .git (gitdir pointer)... and DUPLICATES the root superspec submodule’ — but these 4 specific files look like genuine first-party CI/e2e/validation tooling, vendored upstream content, and predate today’s session (Aug 31 mtime). Per §11.4.124 (investigate-before-remove) and §11.4.122 (no silent removal without operator decision), this needs git-history investigation (was there ever a commit touching these paths? are they meaningful with this project or are they truly part of the vendored nested checkout and should stay ignored?) before either (a) carving a negation exception into .gitignore for exactly these 4 files or (b) confirming they are genuinely disposable vendored artifacts and documenting that explicitly. Acceptance: CM-GITIGNORE-SWALLOW-GUARD passes clean (0 findings) once the 4 files are explicitly, evidence-backed classified as vendored-and-correctly-ignored.
BOB-244	Bug	Queued	Critical	

ATM ID	Type	Status	Severity	Description
BOB-246	Bug	In progress	Critical	with that classification recorded in the .gitignore comment itself. Discovered 2026-09-25 by a subagent implementing 003-zero-shortcomings-audit Task 1, independently reproduced in isolation. script commit-push-all.sh's argument parser stops reading flags at the first non-flag token; when –scope flags are placed AFTER the commit-message positional argument (exactly as tasks.md's own Step 5 template shows for every task in the 003-zero-shortcomings-audit plan, e.g. "bash scripts/commit-push-all.sh "" –scope the –scope flags are silently swallowed as if they were part of the message, no error or warning emitted, and the script falls through to its unconditional/unscoped git add -A path. Real world consequence (2026-09-25): following the exact template ordering caused a scoped Task commit to instead sweep in 5 files – the intention 2, a sibling task's 2 legitimate-but-not-yet-reviewed files, and a DELIBERATELY-uncommitted, known-broken/hanging test file (tests/ddos/test_sse_stream_stress_chaos.py, see BOB-097 diary entry_id=16) – and PUSH all of them to two remote git hosts before the mistake was caught. No data was lost, nothing was force-pushed, but a silent full-tree-sweep on a routine commit is a serious safety gap in a script this project's own constitution names as the ONLY sanctioned commit path. Root cause: the parser flag/positional-arg interleaving logic assumes flags always precede the message and never validates that assumption or warns when scope flags appear post-message. Remediation: (1) make the parser accept –scope flags in ANY position relative to the message (getopts-style two-pass parsing, or explicit flag-then-positional-then-more-flags handling), OR (2) at minimum detect scope-flag-shaped tokens appearing after the first positional argument and REFUSE with a loud error rather than silently falling back to add -A – a parser that cannot tell 'no –scope

ATM ID	Type	Status	Severity	Description
				given, unscoped commit intended' apart from '-scope given but in the wrong position' is its §11.4.201(1) false-negative-class defect. (3) Au and fix tasks.md's own Step 5 example order across every task (flags before the message, matching the working pattern used successfu elsewhere this session) so the plan itself does keep tripping this bug for every future task implementer.